

```

# Import Libraries
# -----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (mean_squared_error, r2_score, accuracy_score, confusion_matrix)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.cluster import KMeans
from matplotlib.gridspec import GridSpec

# -----
# LOAD & PREPARE DATA
# -----
final_df=pd.read_csv("C:/Sure Proed/Climate Analytics/python/Processed_Climate_a

# -----
# STAGE 1 – DEFINE THE PROBLEM
# -----

print("=" * 65)
print("STAGE 1 – DEFINE THE PROBLEM")
print("=" * 65)
print("""
Problem Statement:
    Predict TEMPERATURE ANOMALY (°C) for countries around the world
    based on CO2 emissions, rainfall, year, and region.

Goal:
    → Regression : predict exact anomaly value
    → Classification : predict risk level (Low / Medium / High)
    → Clustering : group similar countries/regions together
Dataset Summary:
""")

print(final_df.describe().round(2))
print(f"\nTotal Records : {len(final_df)}")
print(f"Years Covered : {final_df['year'].min()} – {final_df['year'].max()}")
print(f"Countries      : {final_df['country'].nunique()}")
print(f"Regions          : {final_df['region'].nunique()}")

```

STAGE 1 – DEFINE THE PROBLEM

Problem Statement:

Predict TEMPERATURE ANOMALY (°C) for countries around the world based on CO₂ emissions, rainfall, year, and region.

Goal:

- Regression : predict exact anomaly value
 - Classification : predict risk level (Low / Medium / High)
 - Clustering : group similar countries/regions together
- Dataset Summary:

	year	Temp_Anomaly °C)	CO2_Emissions (MtCO2)	rainfall_mm
count	520.00	520.00	520.00	520.00
mean	2012.50	0.95	1155.81	830.90
std	7.51	0.50	2036.86	484.36
min	2000.00	-0.08	29.70	11.20
25%	2006.00	0.58	278.23	538.48
50%	2012.50	0.88	395.10	664.55
75%	2019.00	1.26	820.18	1086.35
max	2025.00	2.95	12570.70	2406.30

Total Records : 520
Years Covered : 2000 – 2025
Countries : 20
Regions : 6

```
#  
# STAGE 2 – SPLIT DATASET INTO TRAIN / TEST  
#  
print("\n"+"="*65)  
  
print("STAGE 2 – TRAIN / TEST SPLIT")  
  
print("=" * 65)  
  
# Encode region as number for ML models  
le=LabelEncoder()  
final_df["region_enc"]=le.fit_transform(final_df["region"])  
  
# Add Temp_Risk label for classification  
final_df["temp_risk"]=pd.cut(  
    final_df["Temp_Anomaly °C"),
```

```

        bins=[-999, 0.5,1.5,999],
        labels=['Low','Medium','High']
    )
    risk_map = {'Low':0,'Medium':1,'High':2}
    final_df["Risk_label"]=final_df["temp_risk"].map(risk_map).astype(int)

    Features=["year", "CO2_Emissions (MtCO2)","rainfall_mm","region_enc"]
    Target="Temp_Anomaly °C)"

    X=final_df[Features]
    y=final_df[Target]

    X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)

    print(f"\nTotal samples : {len(final_df)}")
    print(f"Training set : {X_train.shape[0]} rows ({X_train.shape[0]/len(final_df)*100}%)")
    print(f"Testing set : {X_test.shape[0]} rows ({X_test.shape[0]/len(final_df)*100}%)")
    print(f"\nTarget column : {Features}")
    print(f"\nTrain y stats : \n{y_train.describe().round(3)}")
    print(f"\nTest y stats : \n{y_test.describe().round(3)}")

```

===== STAGE 2 – TRAIN / TEST SPLIT =====

```

Total samples : 520
Training set : 416 rows (80%)
Testing set : 104 rows (20%)

```

```

Target column : ['year', 'CO2_Emissions (MtCO2)', 'rainfall_mm', 'region_enc']

```

```

Train y stats :
count      416.000
mean         0.958
std          0.488
min         -0.080
25%          0.597
50%          0.900
75%          1.282
max          2.950
Name: Temp_Anomaly °C), dtype: float64

```

```

Test y stats :
count      104.000
mean         0.901

```

```
std      0.556
min      -0.030
25%      0.520
50%      0.800
75%      1.110
max      2.740
Name: Temp_Anomaly °C), dtype: float64
```

```
# -----
# STAGE 3 - RANDOM FOREST REGRESSOR
# -----
```

```
print("\n" + "="*65)
print("STAGE 3 - RANDOM FOREST REGRESSOR")
print("=" * 65)
```

```
rf= RandomForestRegressor(
    n_estimators = 200, # 200 trees used
    max_depth = 10, # max_depth=10: limits tree depth to prevent overfitting
    min_samples_split = 4,
    random_state = 42,
    n_jobs=-1 # n_jobs: uses all CPU cores to speed up training (-1 = full power)
)
```

```
rf.fit(X_train,y_train)
y_pred_rf = rf.predict(X_test)
rmse_rf=np.sqrt(mean_squared_error(y_test,y_pred_rf))
r2_rf=r2_score(y_test,y_pred_rf)
mae_rf=np.mean(np.abs(y_test-y_pred_rf))
```

```
print(f"\n n_estimators : 200")
print(f" max_depth : 10") # max_depth=10: limits tree depth to prevent overfitting
print(f"\n RMSE : {rmse_rf:.4f} °C") # RMSE: average prediction error (penalizes large errors)
print(f" MAE : {mae_rf:.4f} °C") #Average absolute error is 0.15°C
print(f" R² : {r2_rf:.4f} ({r2_rf:.1f}% variance explained)")
```

```
=====
STAGE 3 - RANDOM FOREST REGRESSOR
=====
```

```
n_estimators : 200
max_depth : 10

RMSE : 0.2095 °C
MAE : 0.1520 °C
R² : 0.8568 (0.9% variance explained)
```

```

# -----
# STAGE 4 – LINEAR REGRESSION + COMPARISON
# -----

print("\n" + "="*65)
print("STAGE-4 - LINEAR REGRESSION & MODEL COMPARISON")
print( "=" * 65)

lr=LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
rmse_lr = np.sqrt(mean_squared_error(y_test,y_pred_lr))
r2_lr = r2_score(y_test ,y_pred_lr)
mae_lr = np.mean(np.abs(y_test - y_pred_lr))

print(f"\n Linear Regression Coefficients:")
for feature,coef in zip(Features, lr.coef_):
    print(f" {feature:30s}: {coef:+.6f}")

print(f" Intercept: {lr.intercept_:.4f}")
print(f"\n RMSE : {rmse_lr:.4f}°C") # RMSE: average prediction error
print(f" MAE : {mae_lr:.4f} °C")
print(f" R² : {r2_lr:.4f} ({r2_lr*100:.1f}% variance explained)")

=====
STAGE-4 - LINEAR REGRESSION & MODEL COMPARISON
=====

Linear Regression Coefficients:
year                : +0.046622
CO2_Emissions (MtCO2) : +0.000033
rainfall_mm         : -0.000131
region_enc          : +0.014238
Intercept: -92.8444

RMSE : 0.3379°C
MAE : 0.2370 °C
R² : 0.6274 (62.7% variance explained)

# -----
# STAGE 5 – FEATURE IMPORTANCE
# -----

print("\n" + "=" * 65)

```



```

print("\n" + "=" * 65)
print("STAGE 6 – K-MEANS Clustering (Region Segmentation)")
print("=" * 65)

Scaler = StandardScaler()
cluster_features = ["Temp_Anomaly °C", "CO2_Emissions (MtCO2)", "rainfall_mm"]
X_scaled = Scaler.fit_transform(final_df[cluster_features])

# Elbow method to find optimal k
inertias, silhouettes = [], []
k_range = range(2, 9)
from sklearn.metrics import silhouette_score
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))

# Fit final model with k=4
km4 = KMeans(n_clusters=4, random_state=42, n_init=10)
final_df["cluster"] = km4.fit_predict(X_scaled)

cluster_profile = final_df.groupby("cluster")[cluster_features].mean().round(3)
cluster_profile["year"] = final_df.groupby("cluster")["year"].mean().round().astype(int)
# Count records per cluster
cluster_count = final_df.groupby("cluster")["country"].count().rename("count")
cluster_profile = cluster_profile.join(cluster_count)

# Cluster labels based on profile
cluster_names = {0: "Moderate Warm", 1: "Cool Low-co2", 2: "Hot High-co2", 3: "Wet"}
final_df["Cluster_Name"] = final_df["cluster"].map(cluster_names)

print("\n cluster profiles (mean values):")
print(cluster_profile.to_string())
print("\n cluster Labels:")
for k, name in cluster_names.items():
    count = cluster_count[k]
    print(f" cluster {k} - {name:20s} : {count} records")

# Top countries per cluster
print("\n Top Countries per cluster:")
for k in range(4):
    countries = final_df[final_df["cluster"] == k]["country"].value_counts().head(5)

```

```

print(f" cluster{k} ({cluster_names[k]}): {", ".join(countries)}")

# Elbow curve
inertias = []
k_range = range(2, 9) # define k range
for k in k_range:
    model=KMeans(n_clusters=k,random_state=42,n_init=10)
    model.fit(X)
    inertias.append(model.inertia_)
print(len(k_range), len(inertias))
plt.plot(k_range,inertias,marker='o')
plt.xlabel("Number of clusters(k)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()

```

STAGE 6 – K-MEANS Clustering (Region Segmentation)

```

cluster profiles(mean values):
    Temp_Anomaly °C) CO2_Emissions (MtCO2) rainfall_mm year count
cluster
0          1.537          661.851      636.429 2019    132
1          0.656          461.496      547.871 2009    207
2          1.126       6750.494      663.690 2013     50
3          0.743          615.269     1537.916 2012    131

```

cluster Labels:

```

cluster 0 - Moderate Warm      : 132 records
cluster 1 - Cool Low-co2      : 207 records
cluster 2 - Hot High-co2      : 50 records
cluster 3 - Wet Tropical      : 131 records

```

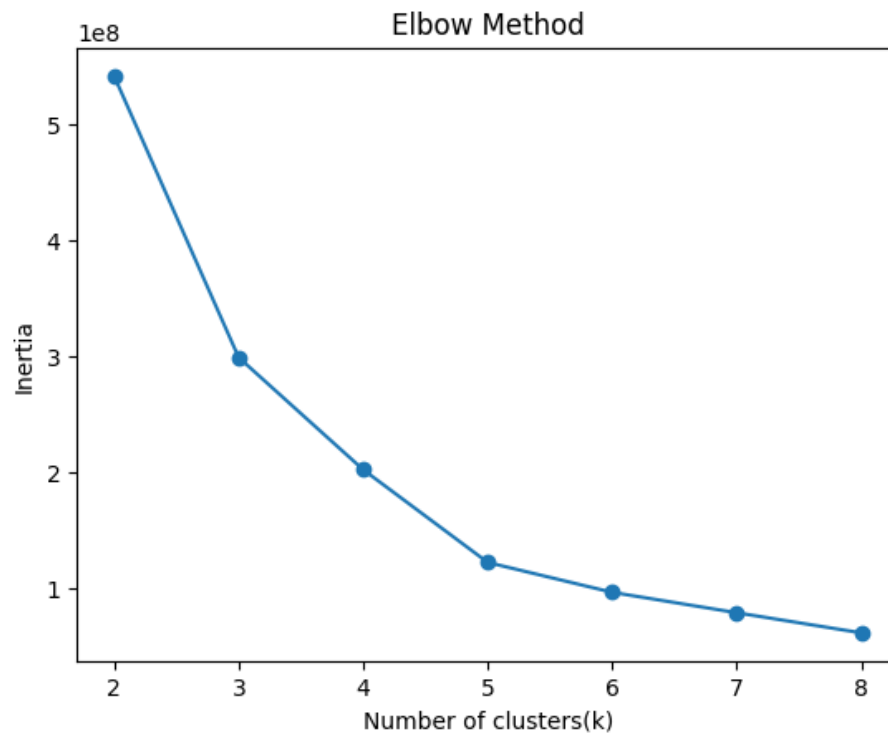
Top Countries per cluster:

```

cluster0 (Moderate Warm): Russia, Canada, Germany
cluster1 (Cool Low-co2): South Africa, Argentina, Egypt
cluster2 (Hot High-co2): USA, China
cluster3 (Wet Tropical): Brazil, Japan, Nigeria

```

7 7



```
# -----  
# STAGE 7 – CLASSIFICATION MODEL (Risk Levels)  
# -----  
print("\n" + "="*65)  
print("STAGE 7 - RISK LEVEL CLASSIFICATION MODEL")  
print("=" * 65)  
  
# 1. Create Risk Label  
# -----  
def get_risk(x):  
    if x < 0.5:  
        return "Low"  
    elif x <= 1.5:  
        return "medium"  
  
    else:  
        return "High"
```

```

# Create temp_risk column

final_df["temp_risk"]=final_df['Temp_Anomaly °C'].apply(get_risk)

# Encode Risk_Label
# -----

le = LabelEncoder()
final_df["Risk_Label"] = le.fit_transform(final_df["temp_risk"])

print("\n Risk Level Distribution:")
print(final_df["temp_risk"].value_counts().to_string())
print(f"\n Low : Temp anomaly < 0.5°C")
print(f" Medium : Temp Anomaly 0.5 -1.5°C")
print(f" High : Temp Anomaly > 1.5°C")

X_clf = final_df[Features]
y_clf = final_df["Risk_Label"]

X_tr_c,X_te_c, y_tr_c, y_te_c = train_test_split(X_clf,y_clf, test_size=0.2, ran

rfc = RandomForestClassifier(
    n_estimators=200,max_depth=10,random_state=42, n_jobs=-1
)

rfc.fit(X_tr_c, y_tr_c) # RandomForestClassifier
y_pred_c = rfc.predict(X_te_c)
y_prob_c = rfc.predict_proba(X_te_c) # Probability
acc= accuracy_score(y_te_c,y_pred_c) # Accuracy_score

print(f"\n Classification Accuracy : {acc:.4f} ({acc*100:.1f}%")
print(f"\n Detailed Report:")
print(classification_report(y_te_c, y_pred_c,
    target_names=['Low', 'Medium', 'High'],zero_division=0))

```

```

=====
STAGE 7 - RISK LEVEL CLASSIFICATION MODEL
=====

```

```

Risk Level Distribution:
temp_risk
medium    352
Low       104
High       64

```

Low : Temp anomaly < 0.5°C
 Medium : Temp Anomaly 0.5 -1.5°C
 High : Temp Anomaly > 1.5°C

Classification Accuracy : 0.8750 (87.5%)

Detailed Report:

	precision	recall	f1-score	support
Low	0.73	0.62	0.67	13
Medium	0.94	0.81	0.87	21
High	0.88	0.94	0.91	70
accuracy			0.88	104
macro avg	0.85	0.79	0.82	104
weighted avg	0.87	0.88	0.87	104

```
# -----
# STAGE 8 – MODEL EVALUATION (Accuracy, RMSE, Confusion Matrix)
# -----
```

```
print("\n" + "=" * 65)
print("STAGE 8 – MODEL EVALUATION")
print("=" * 65)
```

```
cm = confusion_matrix(y_te_c ,y_pred_c)
print(f"\n ==== Regression Metrics ====")
print(f" Linear Regression → RMSE:{rmse_lr:.4f} MAE: {mae_lr:.4f} R²: {r2_lr:.4f}")
print(f" Random Forest → RMSE:{rmse_rf:.4f} MAE: {mae_rf:.4f} R²:{r2_rf:.4f}")
print(f" Improvment → RMSE:(((rmse_lr-rmse_rf)/rmse_lr*100):.1f}% better R²:{{{r2_rf-r2_lr}*100:.1f}% better")
```

```
print(f"\n === Classification Metrics===")
```

```
print(f" Accuracy : {acc:.4f} ({acc*100:.1f}%)"
```

```
print(f"\n Confusion Matrix:")
```

```
print(f"          Predicted")
```

```
print(f"          Low Medium High")
```

```
for i, label in enumerate(['Low', 'Medium', 'High']):
```

```
    print(f" Actual {label} | {" ".join(str(v) for v in cm[i])}")
```

```
=====
```

STAGE 8 – MODEL EVALUATION

==== Regression Metrics ====

Linear Regression → RMSE:0.3379 MAE: 0.2370 R²: 0.6274

Random Forest → RMSE:0.2095 MAE: 0.1520 R²:0.8568

Improvement → RMSE:38.0% better R²:26.8% better

=== Classification Metrics===

Accuracy : 0.8750 (87.5%)

Confusion Matrix:

	Predicted		
	Low	Medium	High
Actual Low	8	0	5
Actual Medium	0	17	4
Actual High	3	1	66

STAGE 9 – INSIGHTS: WHICH REGIONS ARE MOST VULNERABLE?

```
print("\n" + "="*65)
print("  STAGE-9 - INSIGHTS: MOST VULNERABLE REGIONS")
print( "=" * 65)

vulnerability = final_df.groupby(["region", "country"]).agg(
    Avg_Anomaly = ("Temp_Anomaly °C)", "mean"),
    Max_Anomaly = ("Temp_Anomaly °C)", "max"),
    Avg_co2      = ("CO2_Emissions (MtCO2)", "mean"),
    Avg_Rainfall = ("rainfall_mm", "mean"),
    High_risk_percentage = ("Risk_Label", lambda x: (x==2).mean()*100),
    Years_High_risk      = ("Risk_Label", lambda x: (x==2).sum())
).round(3).sort_values("Avg_Anomaly", ascending=False).reset_index()

print("\n Top 10 Most vulnerable Countries")
print(f" {"rank":<5} {"country":<15} {"region":<10} {"Avg°C":>7} {"Max°C":>9} {"")
print(" " + "-" * 65)

for i, row in vulnerability.head(10).iterrows():
    rank = i + 1
    bar = "█" * int(row['High_risk_percentage']/5)
    print(f" {"rank":<5} {"row["country"]":<15} {"row["region"]":<14} "
          f" {"row['Avg_Anomaly']":>7.3f} {"row["Max_Anomaly"]":>7.3f} "
          f" {"row["High_risk_percentage"]":>9.1f}% {"bar}")
```

```

Region_vuln = final_df.groupby("region").agg(
    Avg_Anomaly = ("Temp_Anomaly °C"), "mean"),
    High_risk_percentage = ("Risk_Label", lambda x: (x==2).mean()*100)
).round(3).sort_values("Avg_Anomaly",ascending=False)

print(f"\n Most Vulnerable Regions (by Avg Temp Anomaly):")
print(Region_vuln.to_string())

# Top 10 countries
top10 = vulnerability.head(10)

# Reverse for better top-to-bottom display
Countries = top10["country"][::-1]
Values = top10["Avg_Anomaly"][::-1]

# Create bar chart
fig = plt.figure(figsize=(12,8))
gs = GridSpec(2, 2, figure=fig,hspace=0.5, wspace=0.3)

plot1= fig.add_subplot(gs[0, :])
bars = plot1.barh(Countries,Values)

plot1.set_title('Top 10 Most Vulnerable Countries – Avg Temp Anomaly',fontweight="bold")
plot1.set_xlabel("Average Temperature Anomaly (°C)",fontweight="bold")

# Add values on bars
for bar, val in zip(bars,Values):
    plot1.text(
        bar.get_width(),
        bar.get_y()+bar.get_height()/2,
        f"{val:.2f}°C",
        va="center"
    )

# Region avg anomaly
#-----
# Group data by region and take average

reg_avg = final_df.groupby("region")["Temp_Anomaly °C"].mean()

```

```

# Create Bar Chart
plot2 = fig.add_subplot(gs[1, 0])
bars2 = plot2.bar(reg_avg.index, reg_avg.values,color="powderblue")

# Title and labels
plot2.set_title("Avg Temp Anomaly by Region",fontweight="bold")
plot2.set_ylabel("Temp_Anomaly (°C)",fontweight='bold')

# Rotate x labels for readability
plot2.set_xticklabels(reg_avg.index, rotation=20)

# Add Values On Bars
for bar,v in zip(bars2, reg_avg.values):
    plot2.text(
        bar.get_x() + bar.get_width()/2,
        v,
        f"{v:.2f}",
        ha='center',fontsize=9
    )

# High risk % per region

plot3= fig.add_subplot(gs[1, 1])
reg_risk=final_df.groupby("region").apply(lambda x:(x["Risk_Label"]==2).mean()*100)
bars3 = plot3.bar(reg_risk.index, reg_risk.values,
                  color=plt.cm.Reds(np.linspace(0.3, 0.85, len(reg_risk))),
                  edgecolor='white', width=0.6)

plot3.set_title('% of Years with HIGH Risk per Region', fontweight='bold')
plot3.set_ylabel("% High Risk Years")

# Rotate x labels for readability
plot3.set_xticklabels(reg_risk.index, rotation=20)

for bar, v in zip(bars3,reg_risk.values):
    plot3.text(bar.get_x()+bar.get_width()/2,v + 0.2,f"{v:.1f}%",ha="center" , fontweight='bold')

```

```

=====
STAGE-9 - INSIGHTS: MOST VULNERABLE REGIONS
=====

```

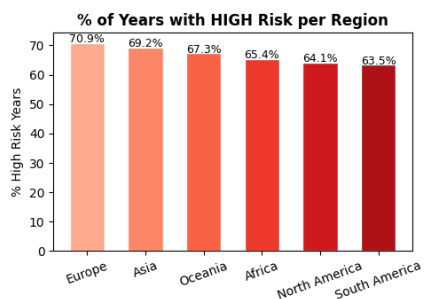
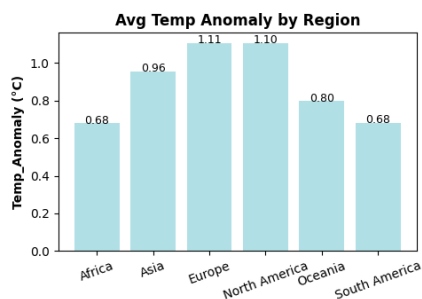
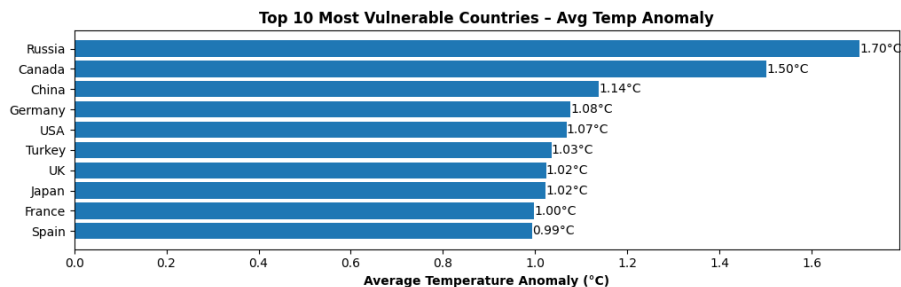
Top 10 Most vulnerable Countries

rank	country	region	Avg°C	Max°C	High_Risk%

1	Russia	Europe	1.704	2.950	46.2%
2	Canada	North America	1.502	2.680	46.2%
3	China	Asia	1.138	1.760	69.2%
4	Germany	Europe	1.077	1.740	84.6%
5	USA	North America	1.068	1.810	73.1%
6	Turkey	Europe	1.035	1.850	73.1%
7	UK	Europe	1.024	2.020	76.9%
8	Japan	Asia	1.023	1.890	80.8%
9	France	Europe	0.998	1.770	76.9%
10	Spain	Europe	0.993	1.750	73.1%

Most Vulnerable Regions (by Avg Temp Anomaly):
Avg_Anomaly High_risk_percentage

region		
Europe	1.108	70.879
North America	1.105	64.103
Asia	0.957	69.231
Oceania	0.797	67.308
South America	0.682	63.462
Africa	0.679	65.385



STAGE 10 – EXPORT FINAL MODEL OUTPUTS FOR POWER BI

```

# -----

print("\n" + "=" *65)
print("Stage 10 - Export Final Output For Power BI")
print("="*65)

# Main export: all predictions + labels
Final= final_df[["year","region","country","Temp_Anomaly °C"],"CO2_Emissions (Mt",
                "temp_risk",'Risk_Label',"cluster","Cluster_Name"]].copy()

Final["RF_Predicted_Anomaly"] = rf.predict(final_df[Features]).round(4)
Final["LR_Predicted_Anomaly"] = lr.predict(final_df[Features]).round(4)
Final["RF_Error"]             = (Final["Temp_Anomaly °C"])-Final["RF_Predicted_A
Final.to_csv("Final_output.csv",index=False)

# Vulnerability
vulnerability.to_csv("vulnerability_summary.csv")

# Cluster profiles
cluster_profile.to_csv("cluster_profile.csv")

# Yearly forecast table -ARIMA
# model accuracy
metrics = pd.DataFrame({
    "Model" : ["Linear Regression", "Random Forest Regressor", "RF Classifier"],
    "RMSE"  : [round(rmse_lr,4),      round(rmse_rf,4),      None],
    "MAE"   : [round(mae_lr,4),       round(mae_rf,4),       None],
    "R2"    : [round(r2_lr,4),        round(r2_rf,4),        None],
    "Accuracy": [None,                None,                round(acc,4)]
})

metrics.to_csv("model_metrics.csv",index=False)

=====
Stage 10 - Export Final Output For Power BI
=====

```